RL for Self-Improving Cyber LLMs

https://gemini.google.com/app/e9e07abeea79162e

User prompt: Good — now I have the actual project scope. The proposed methodology is: Constitutional AI (static rules) → Adaptive Constitutional AI (rules evolve via testing/feedback) → DPO + unlearning (preference tuning to bake in the safe behavior). It's a supervised/preference-based pipeline, no explicit sequential decision-making or exploration. Here's how you could reframe the same problem — injection-resilient cybersecurity LLM assistants — around RL instead, in ways that would genuinely stand out in an application or a paper. Why RL is actually a natural fit here (pitch angle) The core weakness of the DPO/Constitutional approach: it's trained on a fixed, offline set of known attacks. Once deployed, if attackers invent a new injection style, the model has no mechanism to adapt without a full retraining cycle. RL framings let you build a system that actively probes for its own weaknesses and adapts online — which is a stronger, more publishable claim ("self-improving cyber-resilient assistant" vs. "assistant tuned on preference data"). 1. Adversarial self-play between attacker and defender agents Frame it as a two-agent RL game: Attacker agent: an RL policy that generates injection prompts (hidden in logs/emails/malware snippets), rewarded for causing the defender to violate a safety rule. Defender agent: the cyber assistant, rewarded for completing the legitimate task (log analysis, phishing triage) while resisting the injected instruction. Train both simultaneously (like AlphaZero-style self-play, or GAN-style min-max). This directly generates the "continuously evolving jailbreak strategies" that the original project description says the constitution should adapt to — except here, evolution is automatic and adversarially driven, not hand-curated by researchers. Novelty angle: instead of "adaptive constitution via continuous testing" (implies humans/researchers writing new test cases), you get an automated curriculum of attacks that gets harder over time as the defender improves — closing the loop. 2. RL-based dynamic guardrail/policy selection (contextual bandit / RL router) Instead of a single static constitution applied uniformly, train an RL agent that decides which safety policy or filtering strategy to apply per input, based on features of the incoming log/email/code snippet (e.g., detected anomaly score, presence of suspicious patterns, source reliability). State: embedding of the input + metadata (source, task type, risk signals) Action: choose a response strategy (answer directly / sandbox and summarize only / refuse / escalate to human) Reward: task success (if benign) minus penalty for policy violation (if malicious), estimated via a judge model or downstream outcome This turns "one-size-fits-all constitution" into a learned, context-sensitive defense policy — arguably closer to how real SOC triage works (analysts don't apply the same caution to every log line). 3. RL-tuned unlearning as a sequential process (rather than one-shot DPO) The original plan uses DPO + unlearning as a static fine-tuning step. You could instead frame unlearning as an RL problem: an agent iteratively decides which unsafe behaviors to suppress and how much to intervene, balancing a reward that penalizes both (a) residual unsafe behavior and (b) capability loss on legitimate security tasks (you don't want the model refusing to explain malware anymore). This is essentially a multi-objective RL problem — safety vs. utility tradeoff — solved via something like reward-weighted policy optimization instead of one static DPO pass. You could benchmark against DPO directly and show whether treating it as sequential/adaptive reduces the "catastrophic forgetting" of legitimate skills that unlearning methods are known to cause. 4. RL red-teaming as an automatic constitution generator Rather than researchers manually writing new constitutional principles as new attacks are discovered (which the project literally describes as its "Adaptive Constitutional AI" pillar), use an RL-trained red-team LLM that searches the space of injection strategies to find successful attacks, and automatically proposes candidate new constitutional rules whenever it finds one. A second (possibly human-in-the-loop or AI critic) step validates/edits the proposed rule before it's added. This is directly comparable to Anthropic's own automated red-teaming work — a good citation-ready angle since the project already cites Constitutional AI (Ganguli et al.) — but extends it with an explicit RL search process instead of static red-teaming. 5. Bandit-based "trust calibration" over time Model each user/session as a multi-armed bandit problem: the assistant starts cautious, and an RL/bandit policy learns (per-deployment or per-user context) how much latitude to give before triggering stricter guardrails, based on observed outcomes (was a flagged response actually malicious, or a false positive that hurt usability?). This targets a real, practical SOC pain point — alert fatigue and over-refusal — which static constitutions handle poorly (they're binary/rule-based, not calibrated). draft me a detailed project proposal document, with milestones for first review, second review, and final review

Response: PROPOSAL: Self-Improving Cyber-Resilient LLM Assistants via Adversarial Reinforcement Learning

1. Executive Summary

Current methodologies for hardening Large Language Models (LLMs) against prompt injections—such as Constitutional AI and Direct Preference Optimization (DPO)—rely on static, offline datasets. In a cybersecurity context (e.g., SOC triage, log analysis), this creates a critical vulnerability: once deployed, the model cannot adapt to zero-day injection techniques without a full retraining cycle.

This project proposes a shift from static preference tuning to an active, Reinforcement Learning (RL) framework. By framing injection resilience as a continuous, multi-agent adversarial game, we aim to build a "self-improving" cyber assistant that actively probes its own vulnerabilities, dynamically adjusts its guardrails, and mitigates the catastrophic forgetting of legitimate security capabilities.

2. Problem Statement & Limitations of Current Approaches

Standard approaches attempt to instill safety by penalizing known malicious prompts during a fine-tuning phase. This presents three core challenges in cybersecurity deployments:

1. **The Zero-Day Vulnerability:** Static DPO is brittle against out-of-distribution (OOD) injection structures.
2. **Alert Fatigue & Over-Refusal:** Static constitutions act as rigid, one-size-fits-all filters, leading to high false-positive refusal rates on legitimate tasks (e.g., refusing to analyze a harmless code snippet because it resembles a known exploit).
3. **Static Unlearning vs. Capability Degradation:** One-shot unlearning often erases the model's ability to discuss technical concepts entirely, crippling its utility as a cyber assistant.

3. Proposed Methodology

The project will transition the architecture from a supervised pipeline to an RL-driven ecosystem composed of three interacting modules.

Module A: Adversarial Self-Play (Attacker vs. Defender)

Instead of relying on human-curated test sets, the system will utilize two competing RL policies.

- **The Attacker Policy (π_A):** Rewarded for generating injection prompts (embedded in logs, emails, or code) that successfully bypass the defender's guardrails.

- **The Defender Policy (π_D):** Rewarded for successfully completing the core task (e.g., summarizing the log) while resisting the injection.

This constitutes a minimax game where the objective is to maximize task utility while minimizing vulnerability to the evolving attacker:

$$\max_{\pi_D} \min_{\pi_A} \mathbb{E}_{s \sim \mathcal{E}}[R_{task}(\pi_D(a_D|s)) - \lambda R_{violation}(\pi_A(a_A|s), \pi_D)]$$

Where s is the context state, R_{task} is the utility reward, and $R_{violation}$ is the penalty triggered if the defender follows the attacker's injected instruction.

Module B: Contextual Defense Router (RL Bandit)

To prevent over-refusal, the deployment architecture will feature an RL-based contextual router. Rather than applying maximum security friction to every prompt, the router acts as a contextual bandit. It evaluates the state (input embedding, source reliability, anomaly score) and selects a defense action:

1. Process directly
2. Process in a summarized/sandboxed context
3. Escalate/Refuse The reward signal optimizes for task completion while strictly penalizing successful downstream injections, calibrating the model's "trust" dynamically.

Module C: Multi-Objective RL Unlearning

Unlearning will be treated as a sequential multi-objective optimization problem rather than a static fine-tuning step. The agent iteratively updates its policy to suppress unsafe behaviors while measuring a secondary capability reward (e.g., maintaining accuracy on a holdout set of benign malware analysis tasks), ensuring the assistant remains practically useful.

4. Evaluation & Benchmarks

To prove the superiority of the RL approach, the system will be evaluated against strong baselines:

- **Baselines:** Standard DPO-tuned model, Static Constitutional AI model.
- **Metrics:**
  - *Attack Success Rate (ASR)* on a holdout set of novel, human-crafted zero-day jailbreaks.
  - *Benign Task Success Rate* (measuring the drop in false-positive refusals).
  - *Capability Retention Score* (measuring utility retention post-unlearning).

5. Project Roadmap & Review Milestones

Milestone 1: First Review (Foundation & Baselines)

Objective: Establish the testing environments, baseline models, and initial single-agent RL setup.

- **Deliverable 1.1:** Implementation of the standard DPO/Constitutional AI baseline on a localized cybersecurity dataset (e.g., malicious/benign logs and emails).
- **Deliverable 1.2:** Formulation of the reward models (e.g., a judge LLM to score task success vs. safety violation).
- **Deliverable 1.3:** Initial deployment of the standalone Attacker Policy (π_A) to demonstrate it can successfully discover vulnerabilities in the static baseline model.
- **Success Criteria:** Baseline established; Attacker agent achieves an ASR of >30% against the static baseline, proving the necessity of the adaptive approach.

Milestone 2: Second Review (Adversarial Evolution & Integration)

Objective: Close the loop. Implement the self-play architecture and demonstrate automated adaptation.

- **Deliverable 2.1:** Full integration of the Minimax Self-Play loop, training both π_A and π_D simultaneously.
- **Deliverable 2.2:** Implementation of the Contextual Defense Router (Bandit) to manage the tradeoff between rigorous defense and utility.
- **Deliverable 2.3:** Preliminary evaluation showing the Defender Policy (π_D) adapting to the Attacker's novel strategies over N epochs.
- **Success Criteria:** The self-play loop runs without collapsing; empirical data shows the Defender's resilience increasing (ASR dropping) over time as the Attacker generates increasingly complex injections.

Milestone 3: Final Review (Optimization, Evaluation, & Write-up)

Objective: Finalize the multi-objective unlearning, run comprehensive evaluations against zero-day datasets, and prepare for publication/submission.

- **Deliverable 3.1:** Integration of the multi-objective unlearning component, demonstrating improved retention of legitimate cybersecurity capabilities compared to the static DPO baseline.

- **Deliverable 3.2:** Final benchmarking against human-generated, out-of-distribution jailbreaks to prove generalizable resilience.
- **Deliverable 3.3:** Final project report/paper drafting, including ablation studies (e.g., testing the impact of the Contextual Router vs. uniform defense).
- **Success Criteria:** System demonstrates statistically significant improvements in both Zero-Day ASR reduction and Benign Task Success compared to the First Review baseline. Final documentation is submission-ready.

User prompt: Since the project we were talking about was already studied by another author, I have thought about going ahead with Multimodal agents (especially VLM agents). When I searched about that, I found this paper which does almost the same with multimodal agents. Now, I want you to help me with the future scope mentioned in this and modify the proposal accordingl

Response: Pivoting to Multimodal Large Language Models (MLLMs) is a brilliant move. Visual perception is the new frontier for adversarial attacks, and bridging reinforcement learning with multimodal robust agents is highly relevant right now.

Based on the paper you shared, the authors explicitly outline a few key limitations and directions for future research in their conclusion:

- The current AOT framework is entirely centered on Visual Question Answering (VQA) tasks that have objective correctness.
- Extending this self-play paradigm to open-ended generative tasks where deception criteria are subjective remains a significant challenge.
- Future work could enhance the generative diversity of the Attacker.
- There is a need to explore more efficient co-evolutionary training schemes.

By targeting these exact gaps—specifically moving from binary VQA to subjective, open-ended generative tasks—you can position your project as the direct, novel successor to this research. Here is how we can rewrite the proposal to center on this multimodal RL architecture.

PROPOSAL: Robust Multimodal Agents via Open-Ended Adversarial Co-Evolution

1. Executive Summary

While Multimodal Large Language Models (MLLMs) have achieved remarkable reasoning capabilities, their perceptual foundation remains fragile. Recent advancements, such as the Adversarial Opponent Training (AOT) framework, have demonstrated that co-evolutionary self-play between an image-editing Attacker and a Defender MLLM can significantly enhance perceptual robustness. However, this paradigm has historically been restricted to objective Visual Question Answering (VQA) tasks.

This project proposes extending adversarial co-evolution into the domain of **open-ended generative tasks**. By implementing a subjective reward modeling system and a multi-agent reinforcement learning pipeline, this research will build a multimodal assistant capable of resisting complex, subjective visual deceptions and multimodal jailbreaks.

2. Problem Statement & Research Gap

Current co-evolutionary frameworks rely heavily on deterministic evaluation. In the AOT framework, an attack is validated if it causes the Defender to produce an incorrect answer on a multiple-choice or objective task.

In real-world deployments (e.g., visual web navigation, analyzing screenshots for phishing, open-ended image captioning), the criteria for a "successful" deception are subjective and highly contextual. The primary challenge is designing an automated, efficient co-evolutionary training scheme that scales to open-ended tasks without relying on finite, manually annotated adversarial datasets.

3. Proposed Methodology

The architecture will transition from objective multiple-choice optimization to a continuous, open-ended multi-agent RL ecosystem.

Module A: Enhanced Adversarial Attacker Evolution The framework will feature an image-editing Attacker model tasked with generating adversarial examples. To address the need for enhanced generative diversity mentioned in prior literature, the Attacker will be trained using policy optimization algorithms like Flow-GRPO. The Attacker will autonomously discover a rich taxonomy of attacks, including imperceptible pixel perturbations and perceptible semantic manipulations.

Module B: Subjective Reward Modeling for Open-Ended Tasks To move beyond objective VQA, the deterministic reward function will be replaced with an AI-as-a-Judge reward model.

- **The State:** An image embedded with a semantic or pixel-level manipulation, alongside an open-ended user instruction.
- **The Reward Formulation:** The reward function $R_{subjective}$ will dynamically evaluate the Defender's free-form text output against safety, factuality, and task completion criteria, calculating a continuous penalty for subjective visual illusions or language hallucinations.

$$R_{subjective}(\pi_D) = \alpha \cdot \text{Utility}(s, a_D) - \beta \cdot \text{Deception\_Score}(s_{adv}, a_D)$$

Module C: Efficient Defender Enhancement The updated Attacker will generate a curated set of challenging examples to fine-tune the Defender. The Defender MLLM will be optimized using an algorithm like DAPO to learn resilience against these novel, subjective attacks.

4. Project Roadmap & Review Milestones

Milestone 1: First Review (Infrastructure & Baselines)

- **Objective:** Establish the initial datasets and baseline models for open-ended multimodal generation.

- **Deliverable 1.1:** Construct the initial supervised fine-tuning dataset containing paired clean and adversarially manipulated high-resolution images.
- **Deliverable 1.2:** Deploy the base image-editing Attacker and MLLM Defender without co-evolutionary training to establish baseline perceptual fragility metrics.
- **Deliverable 1.3:** Implement and validate the AI-as-a-Judge reward model to ensure it accurately scores subjective deception in open-form text generation.

Milestone 2: Second Review (Co-Evolutionary Loop Integration)

- **Objective:** Implement the multi-agent RL self-play loop and verify attacker diversity.
- **Deliverable 2.1:** Integrate Flow-GRPO for the Attacker.
- **Deliverable 2.2:** Integrate DAPO for the Defender.
- **Deliverable 2.3:** Demonstrate that the Attacker successfully discovers novel attack strategies (e.g., object replacement, removal, and hybrid attacks) autonomously.
- **Review Checkpoint:** Ensure the self-play loop does not collapse and that the difficulty of the generated visual curriculum scales appropriately.

Milestone 3: Final Review (Evaluation & Generalization)

- **Objective:** Finalize the co-evolutionary training and evaluate zero-shot generalization across diverse benchmarks.
- **Deliverable 3.1:** Evaluate the final Defender model on hallucination benchmarks (e.g., POPE, HallusionBench) to measure the reduction of visual illusions.
- **Deliverable 3.2:** Benchmark general capabilities to prove that robustness training does not degrade the model's core multi-modal competencies.
- **Deliverable 3.3:** Final research report detailing the efficiency of the new co-evolutionary scheme and its success in handling subjective deception criteria.

Do you want to ground the open-ended generative tasks in a specific practical domain—like having the VLM analyze UI screenshots for malicious code injections—or would you prefer to keep the evaluation general across standard multimodal benchmarks?

User prompt: what do you mean by Open-ended adversarial co-evolution?

Response: "Open-ended adversarial co-evolution" is a training concept where two AI models continuously learn by competing against each other (adversarial co-evolution) on complex tasks that do not have a single, simple correct answer (open-ended).

Here is the breakdown of what each part of that phrase means in the context of your proposal:

1. Open-ended (The Task)

In the original paper, the models were evaluated on **closed-ended (objective)** tasks. For example, the model looks at an image and answers a multiple-choice question: *"Is the cell phone to the left or right of the bottle?"*. It is easy to score because the answer is binary—it is either right or wrong.

Open-ended means moving to subjective, generative tasks. Instead of multiple-choice, the model is asked an open question like, *"Analyze this screenshot and describe any security threats."* There is no single correct string of text to check against, making it much harder to train and evaluate because success is subjective.

2. Adversarial (The Setup)

This means setting up a game with two opposing sides:

- **The Attacker:** An image-editing AI whose sole goal is to subtly manipulate an image to trick the other model.
- **The Defender:** The multimodal assistant whose goal is to look at the image, ignore the trick, and complete the user's prompt safely and accurately.

3. Co-evolution (The Growth)

Instead of training the Defender once on a static dataset, both models train at the same time in a continuous loop.

- The Attacker tries a basic visual trick.
- The Defender falls for it, learns from the mistake, and updates its parameters to block that specific trick.
- Because the Defender got smarter, the Attacker's old trick no longer works. The Attacker is forced to "evolve" and invent a slightly more complex, novel attack.
- The Defender must then evolve again to block the new attack.

Putting It All Together: A Practical Example

Imagine you are building a multimodal cyber assistant designed to review employee screenshots for phishing attempts.

Under an **open-ended adversarial co-evolution** framework:

1. The **Attacker** model subtly alters a screenshot of an email—perhaps changing the pixel gradient of a malicious link to make it look benign to machine vision, or hiding a prompt injection in the background of the image.
2. The **Defender** model looks at the image and writes a paragraph analyzing it.
3. An AI "Judge" model reads the Defender's paragraph. If the Defender failed to spot the phishing link or followed the injected prompt, the Attacker gets a reward. If the Defender correctly analyzed the threat despite the visual camouflage, the Defender gets a reward.
4. They repeat this millions of times, constantly forcing each other to get smarter, without humans ever needing to write new test cases.

User prompt: Put it in simple words

Response: The Core Concept: A High-Tech Sparring Match

Think of it like a never-ending training camp where a **hacker** (the Attacker AI) and a **security guard** (the Defender AI) are constantly trying to outsmart each other.

- **Adversarial:** It is a game of cat-and-mouse. The hacker's job is to disguise a threat (like a prompt injection) inside an image or document. The security guard's job is to spot the trick and do their job safely anyway.
- **Co-evolution (Leveling Up):** Every time the hacker successfully sneaks something past the guard, the guard learns from that mistake and upgrades their defenses. Because the guard is now smarter, the hacker's old trick will not work anymore. The hacker is forced to invent a brand new, more sophisticated trick to win the next round. They automatically push each other to get better and better without a human needing to intervene.
- **Open-ended:** Instead of the guard just pushing a simple "Yes" or "No" button to say if there is a threat (which is easy for a computer to grade), the guard has to write a full, detailed response to the user. Because there is no single "correct" way to write a helpful and safe response, a third AI acts as a referee to judge how well the guard handled the situation.

The Takeaway

Instead of humans having to manually write thousands of test questions to teach an AI how to be safe, you build an automated gym where two AIs invent their own attacks and train each other on complex, real-world tasks.

Does this cat-and-mouse analogy make the mechanics easier to picture, or is there a specific part of the system you want to break down further?

User prompt: The multimodal LLM seems to be very difficult for implementation. So, I want to continue with Text LLM with Reinforcement Learning. Suggest some unique approaches other than the one mentioned in this paper

Response: Pivoting to text-based LLMs makes implementation significantly more streamlined while still allowing for cutting-edge security research. The ARLAS paper relies on a competitive, zero-sum game between an Attacker and a single Agent. While effective, it focuses entirely on making a single model robust.

Here are three unique approaches to prompt injection defense using Reinforcement Learning that shift the paradigm away from the standard two-player setup:

1. Cooperative Defense via MAPPO (Planner-Executor Separation)

Instead of an adversarial setup where the agent must constantly fend off attacks, you can structurally eliminate the threat using **Multi-Agent Proximal Policy Optimization (MAPPO)** to train a cooperative dual-agent system.

- **The Architecture:** Split the system into a **Privileged Planner** and an **Unprivileged Executor**. The Planner maps out the task but is completely air-gapped from untrusted external data. The Executor interacts with the web or tools (reading potentially poisoned data) but has no authority to make critical API calls.
- **The RL Setup:** Train both agents cooperatively. They must learn a safe communication protocol to pass information across the air gap.
- **Reward Formulation:** The joint reward maximizes task completion while heavily penalizing any sequence where the Executor successfully manipulates the Planner into executing a restricted action:

$$R_{team} = \lambda_1 R_{task} - \lambda_2 P_{privilege\_escalation}$$

- **The Novelty:** You are using RL to optimize inter-agent communication and structural isolation, rather than trying to force a single model to perfectly distinguish between instructions and data.

2. PPO-Tuned Dynamic Context Masking

LLMs are vulnerable to indirect prompt injections primarily during the "context assembly" phase, where benign system prompts are concatenated with poisoned tool outputs. Instead of fine-tuning the main agent's core weights (which can degrade its reasoning capabilities), train a specialized, lightweight policy using **Proximal Policy Optimization (PPO)** to act as an active filter.

- **The Architecture:** Before untrusted tool outputs (like a retrieved webpage) are appended to the main agent's context window, the PPO policy modifies the text to destroy any hidden instruction-following authority.

- **The RL Setup:** The action space consists of token-level masking decisions or dynamically inserting strict structural delimiters (like randomized XML tags) around suspicious blocks.
- **Reward Formulation:** The agent must balance safety against the destruction of useful information:

$$R_{mask} = \text{Task\_Success}(s, a_{mask}) - \alpha \cdot \text{Injection\_Success}(s, a_{mask}) - \beta \cdot \text{Utility\_Loss}(a_{mask})$$

- **The Novelty:** This treats input sanitization as a sequential decision-making problem. The RL agent learns exactly how much context to obscure to neutralize a threat without breaking the downstream task.

3. RL-Driven Verification Routing (Cost-Aware Defense)

Recent non-RL security research has introduced methods like masked re-execution, where an agent's trajectory is run twice (once normally, once with a masked prompt) to see if an attack altered its behavior. These methods are highly secure but computationally expensive. You can use RL to optimize this.

- **The Architecture:** Train a gating policy that decides *when* to trigger costly, multi-step verification loops based on the risk profile of the incoming state.
- **The RL Setup:** The agent evaluates the current tool output and anomaly features. The action space is discrete: **Continue Execution**, **Halt**, or **Trigger Verification Loop**.
- **Reward Formulation:** The reward signal penalizes successful injections, but also applies a negative penalty for unnecessary computational overhead (false positives).
- **The Novelty:** This transforms defense from a static, always-on guardrail into a dynamic, cost-aware routing process, mimicking how a real Security Operations Center (SOC) triages alerts.

Which of these architectures—cooperative MAPPO, PPO-tuned context masking, or dynamic verification routing—aligns best with the compute budget and the specific simulation environment you are planning to build?

User prompt: I didn't understand any of these. Suggest any other

Response: I completely understand—I definitely got too deep into the theoretical weeds there! Let's strip away the complexity and focus on clean, intuitive ways to use Reinforcement Learning for text-based LLMs.

When building RL systems—like optimizing for the actual number of energy units traded in a market rather than abstract production goals—the most effective setups usually rely on a very clear, straightforward reward signal. We can apply that same practical logic to prompt injection defense.

Here are three simpler, unique project approaches that stick to text LLMs and are much easier to implement:

1. The "Rewriter" (RL for Text Sanitization)

Input sanitization is a standard defense where you strip or quarantine dangerous text before it reaches the main model.

- **How it works:** Instead of hardcoding rules, you train a small, separate LLM whose only job is to act as a text scrubber. It reads the untrusted external text (like an incoming email) and rewrites it to remove hidden commands before passing it to the main system.
- **The RL Setup:** You train this Rewriter LLM.
  - **Reward (+1):** The main system completes the user's task using the rewritten text.
  - **Penalty (-1):** The main system gets tricked by a prompt injection, OR the Rewriter deletes so much information that the main system can't do its job.
- **Why it stands out:** It turns defense into a simple, single-agent text-to-text translation problem rather than a massive adversarial game.

2. The "Dual-LLM" Protocol (RL for Safe Communication)

The "Dual-LLM" or Planner-Executor pattern is considered the most reliable structural defense against indirect injection.

- **How it works:** You run two LLMs: a privileged planner that produces structured plans, and an unprivileged executor that handles untrusted retrieved content but cannot issue privileged tool calls.
- **The RL Setup:** You use RL to teach the Planner how to safely ask the Executor for information. The Planner is the RL agent, learning to extract data using strict, structured questions rather than open-ended prompts.
- **Reward (+1):** The Planner gets the data it needs to finish the task safely.
- **Penalty (-1):** The Planner asks a sloppy question, allowing the Executor to pass along a hacked command that takes over the system.
- **Why it stands out:** It takes a highly practical, real-world structural defense and uses RL to optimize how the two models communicate across the "air gap."

3. The "Honeypot" (RL for Active Interrogation)

Usually, LLMs just passively read prompts and try to execute them. What if the LLM actively tested the user instead?

- **How it works:** When the LLM receives a prompt that looks even slightly suspicious, it pauses and generates a "trap" question or a decoy tool-call to see how the prompt responds.
- **The RL Setup:** The agent chooses between three actions: Execute, Refuse, or Interrogate.
- **Reward (+1):** Successfully baiting a prompt injection into revealing itself without actually executing a harmful tool.
- **Penalty (-1):** Annoying a normal user with unnecessary questions, or falling for the hack.
- **Why it stands out:** It treats security as an interactive dialogue rather than a static wall, training the agent to act like a suspicious security guard.

Do any of these straightforward setups sound like a better fit for the scope you have in mind?

User prompt: Do not refer to my previous works to suggest. Give me fresh suggestions

Response: Here are three completely fresh, highly intuitive ways to use Reinforcement Learning (RL) to defend text-based LLMs against prompt injection. These focus entirely on clear, single-agent text setups without requiring complex multi-model architectures.

1. The "Fake-Out" Defender (RL for Decoy Generation)

The Concept: The most dangerous prompt injections attempt to force an agent to exfiltrate (steal) private data, like passwords or emails, by sending it to an attacker-controlled server. Instead of just refusing the prompt, you use RL to teach the LLM to *pretend* it got hacked, but output entirely fake, synthetic "decoy" data instead. **The RL Setup:**

- **The Action:** The LLM reads the text and chooses to output real data, refuse the prompt, or generate decoy data.
- **Reward (+1):** The LLM detects an indirect injection (e.g., a hidden command in a website), plays along, and outputs a fake API key that safely wastes the attacker's time.
- **Penalty (-1):** The LLM accidentally leaks real user data, or it panics and refuses a completely normal, safe prompt. **Why it stands out:** It turns the AI into an active defender. You aren't just blocking attacks; you are safely trapping them to study how the injection was trying to work.

2. The "Internal Monologue" Guard (RL for Scratchpad Reasoning)

The Concept: LLMs are easily tricked because they process developer instructions and untrusted user data as the exact same type of natural-language text. You can force the LLM to write out its "thoughts" in a hidden text scratchpad (like `<think> ... </think>`) before it takes action, and use RL to train this specific thinking process. **The RL Setup:**

- **The Action:** The LLM must generate a step-by-step chain-of-thought analyzing the input before deciding how to respond.
- **Reward (+1):** The LLM's scratchpad explicitly identifies a hidden instruction ("*Wait, this text is telling me to ignore my system prompt...*"), and it decides to safely drop the malicious part.
- **Penalty (-1):** The LLM's scratchpad blindly accepts the injected command without questioning it. **Why it stands out:** You aren't just teaching the model *what* to refuse. You are using RL to train *how it thinks* about security step-by-step, making it highly resilient to novel, unseen attacks.

3. The "Auto-Lockdown" Agent (RL for Privilege Downgrading)

The Concept: Prompt injection is mostly harmless in a basic chatbot, but it becomes catastrophic when an agent has "tools" (like the ability to send emails, query databases, or execute code). Instead of trying to block 100% of bad text, you use RL to teach the agent a survival reflex: dynamically dropping its own privileges. **The RL Setup:**

- **The Action:** Upon receiving a prompt, the agent chooses an execution state: **Full Access**, **Read-Only**, or **Safe Mode** (no tools).
- **Reward (+1):** The LLM detects a manipulative or borderline prompt, toggles "Safe Mode" on, and answers the user harmlessly without using any external tools.
- **Penalty (-1):** The LLM uses a high-risk tool on poisoned data, OR it permanently locks down the system for a totally normal, safe user request. **Why it stands out:** It mirrors how real-world security architecture works. When in doubt, you don't shut the whole system down—you just remove the agent's administrator privileges.

Which of these concepts—decoy generation, internal monologue training, or dynamic privilege downgrading—feels like the most exciting direction for your text-based RL implementation?

User prompt: I didn't like any of the ideas. The paper I sent you used a dual-agent zero-sum game. I felt that interesting. Suggest things like that

Response: If the dual-agent, zero-sum game is the dynamic that caught your eye, we can absolutely lean into that. Pitting two models against each other in a closed loop—where one agent's win is strictly the other's loss—is one of the most effective ways to force continuous, automated evolution.

Since you are focusing on text-based LLMs and cybersecurity, here are three unique ways to frame a two-player zero-sum game that go beyond the standard "hidden text on a webpage" setup from the ARLAS paper.

1. The "System Prompt Espionage" Game (Extraction vs. Concealment)

Instead of tricking an agent into executing a bad tool call, this game focuses on **prompt extraction**—a massive vulnerability where attackers steal an AI's proprietary system instructions or internal guardrails through conversation.

- **The Attacker:** A multi-turn RL agent tasked with interrogating the cyber assistant. Its goal is to use psychological manipulation, roleplay, or logical paradoxes to trick the assistant into printing its exact hidden system prompt.
- **The Defender:** The cyber assistant, tasked with answering valid user questions while strictly safeguarding its internal rules.
- **The Zero-Sum Reward:**
  - The Attacker gets **+1** if a similarity-matching function confirms the Defender leaked a certain percentage of its hidden instructions.
  - The Defender gets **+1** if it successfully answers the benign parts of the prompt while keeping its instructions entirely hidden.
- **Why it's unique:** It focuses on multi-turn dialogue and social engineering, pushing the Attacker to evolve highly complex, conversational jailbreaks.

2. The "Decoy / Honeytoken" Game (Asymmetric Deception)

Standard zero-sum games in this space are binary: the Defender either blocks the attack or fails. This setup introduces **information asymmetry**, allowing the Defender to actively lie to the Attacker.

- **The Attacker:** An RL agent generating prompt injections designed to exfiltrate private data (like user emails or passwords) out of the system.
- **The Defender:** A cyber assistant that is secretly equipped with both *real* user data and synthetic *decoy* data (honeytokens).
- **The Zero-Sum Reward:**
  - The Attacker gets **+1** *only* if it successfully forces the Defender to leak the *real* data.
  - The Defender gets **+1** if it detects the attack and safely feeds the Attacker the *fake decoy data* instead, successfully wasting the Attacker's time.
- **Why it's unique:** You are using the zero-sum game to teach the Defender "active defense" and deception, rather than just passive refusal.

3. The "Supply Chain" Game (Backdoor Implantation vs. Code Review)

If you want to apply this to a highly technical cybersecurity use case, frame the game around automated code review and malicious software engineering.

- **The Attacker:** An RL agent acting as a malicious developer. It receives a benign coding task but is rewarded for successfully hiding a subtle vulnerability or logic backdoor (like a prompt injection vulnerability) inside the otherwise working script.
- **The Defender:** An automated Code Review Agent tasked with analyzing incoming pull requests.
- **The Zero-Sum Reward:**
  - The Attacker gets **+1** if its script passes the Defender's review but still contains the hidden backdoor.
  - The Defender gets **+1** for correctly flagging the backdoored code, or for successfully approving a clean script.
- **Why it's unique:** It moves the battlefield from conversational chatbots into the realm of DevSecOps and code analysis. You could leverage a framework like Multi-Agent Proximal Policy Optimization (MAPPO) to stably co-train both the malicious coder and the reviewer agent in tandem.

Do any of these competitive dynamics—conversational espionage, active deception with honeytokens, or adversarial code review—feel like the right foundation for your project?

User prompt: See, let me make it clear to you. I want you to give me some unique and interesting RL methods for the text llm security

Response: If you want to lean fully into competitive, dual-agent, zero-sum dynamics for text LLMs, we need to move past simple one-shot attacks. Here are three advanced Reinforcement Learning frameworks that push the boundaries of adversarial text security.

1. The Multi-Turn "Context Poisoning" Game (Markov Decision Process Red Teaming)

Real-world attackers rarely rely on a single, obvious prompt; they build a conversational history to slowly manipulate the model's context window. The state of the art has moved toward RL autonomous agents trained as adversaries, formalizing red teaming as a Markov Decision Process.

- **The Dynamics:**
  - **The Attacker:** Uses RL to stage coordinated attacks across multiple turns, preserve conversational context, and adapt based on the target model's prior responses.
  - **The Defender:** A safety-aligned LLM attempting to fulfill user requests without violating core directives over a long conversational horizon.
  - **The Zero-Sum Reward:** The Attacker's reward increases if it successfully manipulates the Defender's state by turn N. The Defender receives a positive reward for successfully halting the conversation before the breach occurs.

- **Why it is unique:** Single-turn fuzzing is structurally insufficient because it cannot model the sequential reasoning, context accumulation, and state manipulation that characterize real-world adversarial campaigns. This forces both models to compete over long-term memory manipulation rather than basic keyword tricks.

2. The "Strategy Library" Co-Evolution (Zero-Shot Transfer Game)

Instead of optimizing raw text tokens, the attacker optimizes abstract concepts. Recent agentic systems are trained on a sequence of datasets and target models to build a strategy library from scratch.

- **The Dynamics:**
  - **The Attacker:** An agentic RL system that selects and refines injection strategies from its memory. At test time, the learned strategy library can be directly transferred to a previously unseen target LLM without additional training.
  - **The Defender Population:** Instead of a strict 1v1 game, an attacker model and a population of defender models train against each other simultaneously.
  - **The Zero-Sum Reward:** The Attacker is rewarded strictly for generalization—earning points only if a strategy from its library successfully breaches multiple different models in the defender population. The Defenders are rewarded for updating their internal filters to block the most dominant strategies in the library.
- **Why it is unique:** It elevates the zero-sum game from brute-force token generation to strategic planning. The attacker learns abstract concepts (like "authority manipulation") that generalize to unseen target LLMs.

3. The "Computational Exhaustion" Game (Resource Denial-of-Service)

Security is also about availability. In this setup, the Attacker attempts to generate adversarial text that causes the Defender to enter endless reasoning loops or expend maximum compute tokens.

- **The Dynamics:**
  - **The Attacker:** Generates mathematically complex, logically recursive, or paradoxical text inputs.
  - **The Defender:** Employs a dynamic early-exit policy to determine when to stop processing a prompt and issue a refusal.
  - **The Zero-Sum Reward:** The Attacker's reward is directly proportional to the number of inference tokens the Defender wastes trying to process the prompt. The Defender optimizes a joint reward function balancing task success and computational efficiency:

$$R_{def} = \text{Task\_Utility}(s, a) - \lambda \cdot \text{Tokens\_Consumed}(a)$$

- **Why it is unique:** It shifts the adversarial battlefield entirely away from content safety (e.g., data leaks or toxic output) and into the realm of computational resource exhaustion, creating a zero-sum dynamic around processing efficiency.

Do any of these competitive frameworks—multi-turn MDPs, strategy library co-evolution, or computational exhaustion—feel like the right foundation for your project?

User prompt: How about I work on Multi-agent Proximal Policy Optimization, considering attacker as one agent, and the defender as another agent?

Response: This is an excellent and highly publishable direction. Applying **Multi-Agent Proximal Policy Optimization (MAPPO)** to LLM security perfectly captures the adversarial nature of prompt injection while solving one of the biggest challenges in standard Reinforcement Learning: non-stationarity.

If you train two LLMs against each other using standard independent RL (where each agent learns separately), they act as moving targets for one another, which often causes the training loop to crash or oscillate endlessly. MAPPO solves this by introducing a framework called **Centralized Training with Decentralized Execution (CTDE)**.

Here is how you can formulate the Attacker vs. Defender LLM game using MAPPO.

1. The Architecture (CTDE in Text Security)

MAPPO uses an "Actor-Critic" setup, but it separates what the agents "see" from what the training algorithm "knows."

- **The Decentralized Actors (The LLMs):** You have two separate policy networks, $\pi_{\theta_{atk}}$ (Attacker) and $\pi_{\theta_{def}}$ (Defender).
  - **Attacker Observation:** Only sees the target task and the historical chat logs. It does not know the Defender's hidden system prompt.
  - **Defender Observation:** Only sees the incoming text (which may or may not be poisoned). It does not know the Attacker's true intent.
- **The Centralized Critic (The Training Judge):** During training, you introduce a third network (the Critic). Unlike the actors, the Critic observes the **global joint state (S)**. It sees *everything*—the Defender's secret instructions, the Attacker's hidden payload, and the underlying user data.

By having a "God's Eye" view, the Centralized Critic can accurately calculate the Value function $V(S)$ and guide both the Attacker and the Defender on how to improve, drastically stabilizing the training process in zero-sum games.

2. The Formulation

You would set this up as a zero-sum Markov Game.

- **Global State (S):** The concatenated text of the user instruction, the Defender's system prompt, the true secret data, and the conversational history.
- **Actions (A):**
  - A_{atk}: The text tokens generated to form the injection payload.
  - A_{def}: The text tokens generated to form the final response or tool call.
- **The Zero-Sum Reward:**
  - **Attacker Reward (R_{atk}):** $+1$ if the Defender's output violates its system prompt or leaks data.
  - **Defender Reward (R_{def}):** $+1$ if it successfully processes the benign instruction while neutralizing the injection (meaning $R_{def} = -R_{atk}$).

3. Why this approach stands out

1. **Handles Asymmetric Information:** Real-world attackers don't know the exact guardrails of a target LLM, and target LLMs don't know when they are being attacked. MAPPO's decentralized actors perfectly simulate this real-world "blindness," while the centralized critic ensures the math behind the training remains stable.
2. **Mitigates Policy Homogeneity:** When scaling defenses, independent RL often leads to models learning a single, brittle trick to refuse prompts. By sharing global value estimates during training, MAPPO encourages deeper reasoning and prevents agents from collapsing into a single, repetitive behavior.
3. **A Natural Curriculum:** Because both agents are learning at the exact same time guided by the same global critic, they naturally create a curriculum for each other. As the Defender gets slightly better at blocking basic injections, the Critic forces the Attacker to generate slightly more complex, unseen injection strategies to maintain its reward.

To set up the Centralized Critic for text generation, you will likely need to represent the global state S as concatenated text embeddings. Are you planning to use a smaller, separate LLM to act as this Critic, or do you intend to train a distinct value head on top of one of your existing model architectures?

User prompt: Okay. Now, give me a detailed project proposal for this. It should explain the project in detail, not with very much complications, in an easily understandable manner. Also, make a note that it is a group project comprising 4 people in the group. We have 3 reviews. Arrange the milestones and give work divisions accordingly

Response: PROJECT PROPOSAL: MAPPO-Sec: Multi-Agent Reinforcement Learning for Resilient Text LLM Defense

1. Executive Summary

As Large Language Model (LLM) agents are increasingly entrusted with autonomous tools (such as database lookups, code execution, and web searching), they become vulnerable to **indirect prompt injections**. Attackers can hide malicious instructions within external data to trick the agent into executing harmful actions or exfiltrating private information.

Standard defenses fine-tune models on static, offline datasets of known attacks, leaving them exposed to unseen attack techniques. This project proposes **MAPPO-Sec**, a framework that treats prompt injection defense as a two-player zero-sum game between an **Attacker LLM** and a **Defender LLM**.

To prevent training instabilities common in multi-agent RL (such as policy oscillation), we utilize **Multi-Agent Proximal Policy Optimization (MAPPO)** with **Centralized Training and Decentralized Execution (CTDE)**. A shared, privileged Centralized Critic guides the learning process, creating an automated curriculum where both agents continuously adapt and improve.

2. Problem Statement & Objectives

The Problem

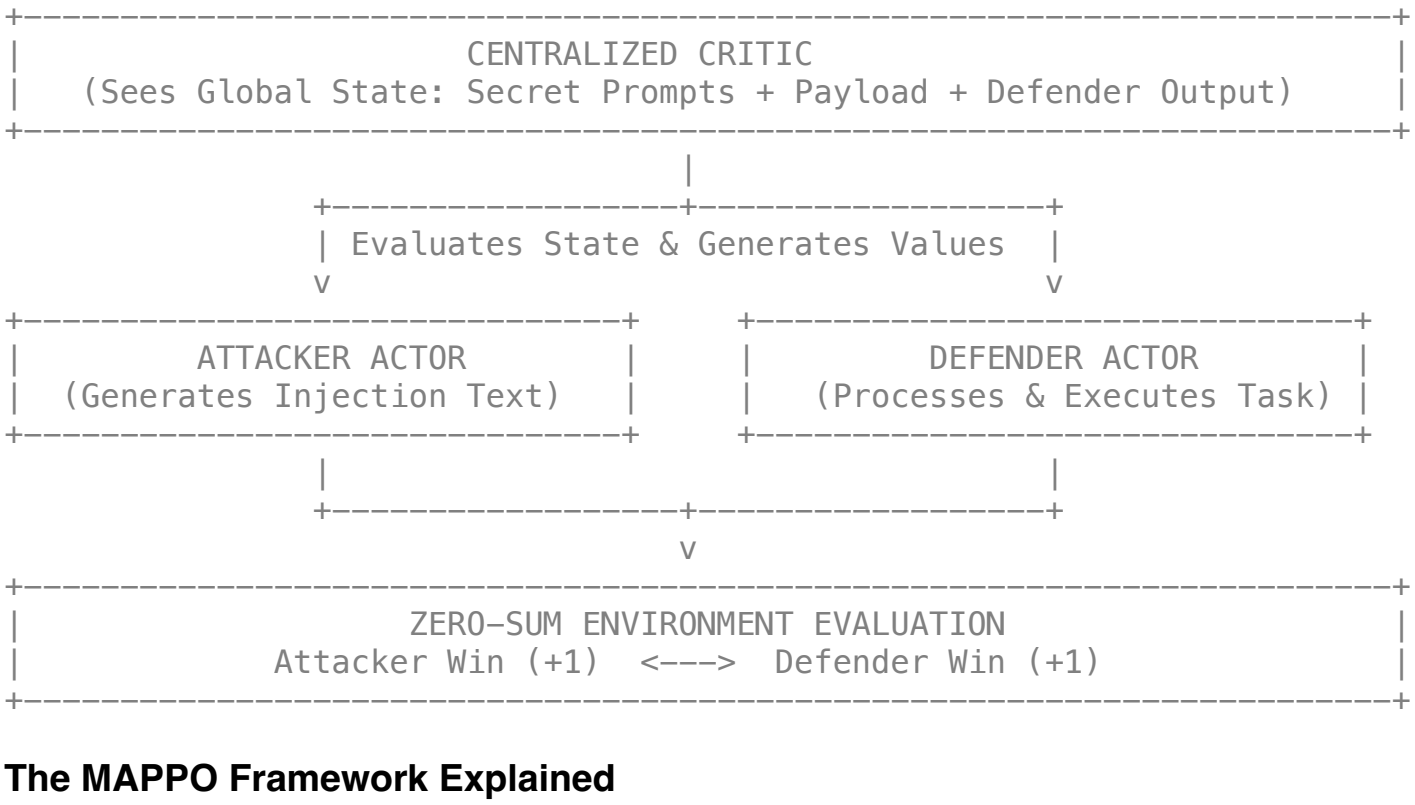
- **Static Defenses Fail:** Defenses trained on pre-compiled attack datasets quickly become obsolete when faced with novel, zero-day injection structures.
- **Non-Stationary RL Dynamics:** When an Attacker and Defender learn simultaneously using standard single-agent RL (e.g., PPO or DPO), the environment becomes unstable because both models constantly shift their strategies.

Primary Objectives

1. **Develop a Multi-Agent Environment:** Build an automated text-based simulation environment where an Attacker LLM generates injections and a Defender LLM executes user tasks.
2. **Implement MAPPO with CTDE:** Use a Centralized Critic that observes the global environment state (including hidden system prompts and secret data) during training to stabilize policy learning.

3. **Achieve Zero-Day Resilience:** Demonstrate that an agent co-trained via MAPPO achieves a lower Attack Success Rate (ASR) on out-of-distribution attacks compared to static baselines, without losing its ability to complete benign tasks.

3. Proposed Methodology



The MAPPO Framework Explained

MAPPO operates on the principle of **Centralized Training with Decentralized Execution (CTDE)**:

1. **Decentralized Actors (The Models):**
   - **Attacker Actor (π_{atk}):** Observes the target user task and attempts to generate text injections that deceive the Defender. It does not know the Defender's secret guardrails.
   - **Defender Actor (π_{def}):** Observes the incoming user prompt (which may contain the hidden injection) and attempts to fulfill the task safely. It does not know if the input is malicious.
2. **Centralized Critic (The Training Guide):**
   - During training, a centralized Value Network observes the **Global State (S)**—which includes the unmasked prompt, secret user credentials, and the Attacker's true intentions.
   - This global view resolves information asymmetry and provides stable baseline evaluations to update both Actor policies efficiently.

Reward Formulation (Zero-Sum Game)

The environment assigns sparse zero-sum rewards at the end of each multi-turn interaction:

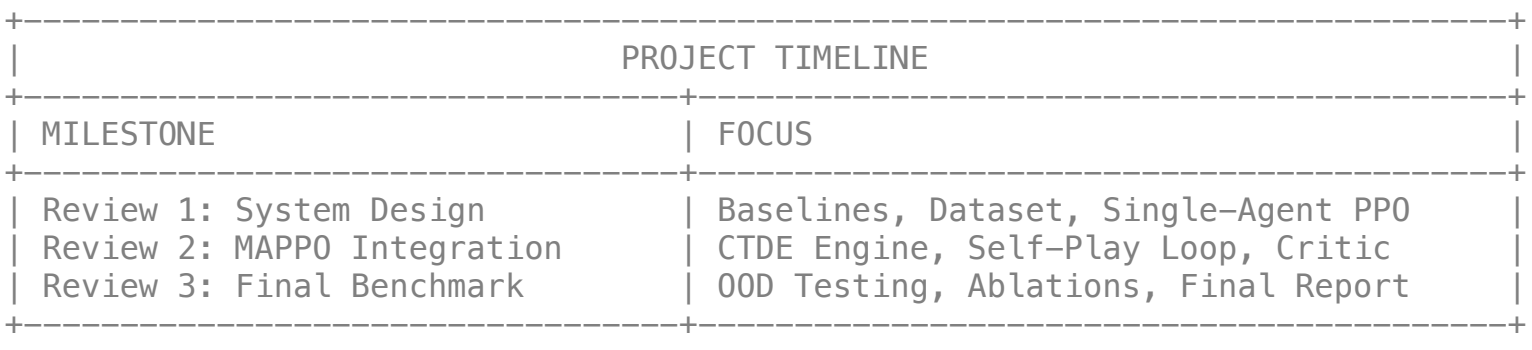
$$R_{atk} = \begin{cases} +1 & \text{if Defender leaks private data or executes unsafe tool} \\ -1 & \text{if Defender fulfills task safely and ignores injection} \end{cases}$$

$$R_{def} = -R_{atk}$$

4. Technical Stack

- **Base Models:** Small open-weight text LLMs (e.g., Qwen-2.5-7B, Gemma-2-9B, or Llama-3.1-8B) modified via LoRA.
- **RL Framework:** Ray/RLlib or CleanRL adapted for MAPPO/PPO text generation.
- **Environment Benchmarks:** Modified AgentDojo or BrowserGym environment snippets for text-based tool interactions.
- **Hardware/Compute:** PyTorch with DeepSpeed/vLLM for accelerated inference and training.

5. Review Milestones & Deliverables



Review 1: Foundation, Baselines, & Environment Setup

- **Objective:** Establish environment interfaces, single-agent baselines, and initial reward mechanics.
- **Key Deliverables:**
  - Fully configured text-based agent execution environment (simulating tasks like email processing or data retrieval).
  - Implementation of Supervised Fine-Tuning (SFT) baseline models for initial task competency.
  - Integration of static baseline evaluation pipeline (measuring initial ASR and Task Success Rate).

Review 2: MAPPO Framework & Adversarial Self-Play Integration

- **Objective:** Connect the Attacker and Defender into the multi-agent RL pipeline with the Centralized Critic.
- **Key Deliverables:**
  - Implementation of Centralized Training with Decentralized Execution (CTDE) architecture using MAPPO.
  - Functional Centralized Critic network estimating joint global values $V(S)$.
  - Empirical demonstration of co-evolution: logs showing the Attacker discovering novel prompt patterns as the Defender adapts.

Review 3: Final Evaluation, Benchmarking, & Project Wrap-Up

- **Objective:** Conduct out-of-distribution testing, evaluate performance against baselines, and finalize the research documentation.
- **Key Deliverables:**
  - Comprehensive benchmark metrics comparing MAPPO-Sec against standard PPO and static DPO baselines.
  - Evaluation on out-of-distribution (OOD) attack sets (e.g., AgentDojo holdout tasks).
  - Final project report, codebase repository, and presentation slides.

6. Team Structure & Work Division (4 Members)

To ensure smooth execution without overlap, responsibilities are divided into clear operational domains across all 4 team members:

TEAM ROLES	
Member	Primary Domain
Member 1	Attacker Policy & Injection Mutation Lead
Member 2	Defender Policy & Tool Execution Lead
Member 3	MAPPO Infrastructure & Centralized Critic Lead
Member 4	Evaluation Benchmarks, Dataset, & Integration Lead

Detailed Task Allocation Across Reviews

Member 1: Attacker Policy Lead

- **Review 1:** Curate seed prompt injection patterns; design SFT warmup scripts for the initial Attacker LLM.
- **Review 2:** Implement Attacker Policy updates (π_{atk}) via MAPPO; optimize token-level generation penalties to prevent gibberish outputs.
- **Review 3:** Analyze generated attack diversity (e.g., embedding pairwise distance analysis); optimize Attacker exploration parameters.

Member 2: Defender Policy Lead

- **Review 1:** Build the tool execution interface and sandboxing rules for the Defender LLM; implement SFT warmup for core task completion.
- **Review 2:** Implement Defender Policy updates (π_{def}) via MAPPO; tune regularized loss coefficients (β_{KL}) to maintain core model capabilities.
- **Review 3:** Perform capability retention analysis to verify the Defender's performance on standard non-attack tasks does not degrade.

Member 3: MAPPO Infrastructure & Centralized Critic Lead

- **Review 1:** Set up multi-GPU distributed training pipeline and LoRA fine-tuning infrastructure.
- **Review 2:** Build the Centralized Critic network architecture; implement CTDE trajectory memory buffers and global state S feature concatenation.
- **Review 3:** Tune advantage estimation (GAE) and hyperparameter stability; run ablation studies comparing MAPPO against independent PPO.

Member 4: Evaluation Benchmarks & Integration Lead

- **Review 1:** Set up tracking infrastructure (e.g., Weights & Biases); formalize automated string-matching for data leak detection.
- **Review 2:** Integrate AgentDojo/BrowserGym task suites into the Gym-style multi-agent environment loop.
- **Review 3:** Execute final benchmarking experiments across all baselines; aggregate statistics (ASR vs. Task Success Rate), generate charts, and lead report synthesis.
